



ClepshydraBot

Resoconto Tecnico — Stato del Progetto

Versione 1.0.0 • Giugno 2026
Autore: tdavide • Confidenziale

INFRASTRUTTURA

Provider	Oracle Cloud Infrastructure
Regione	eu-milan-1 (Milano, Italia)
Shape	VM.Standard.E2.1.Micro
CPU / RAM	1 OCPU • 1 GB RAM
Sistema	Canonical Ubuntu 24.04 LTS
IP Pubblico	80.225.91.142

Indice dei Contenuti

1. Panoramica del Progetto
2. Infrastruttura e Ambiente di Esecuzione
3. Stack Tecnologico
4. Struttura del Repository
5. Funzionalità Implementate
6. Sistema di Cache e Dati Persistenti
7. Interazione con API Esterne
8. Flusso di una Verifica Mazza Artisan
9. Limitazioni Attuali e Debito Tecnico
10. Architettura V2 (Layer + Tournament)
11. Roadmap — ClepshydraBot V2

1. Panoramica del Progetto

ClepshydraBot è un bot Discord sviluppato in Python per gestire una community dedicata a Magic: The Gathering Arena. Il bot automatizza tre aree principali: il censimento e la verifica degli utenti al loro ingresso nel server, la validazione dei mazzi per il formato Artisan tramite l'API di Scryfall, e la generazione di immagini personalizzate per il showcase dei mazzi verificati.

Il progetto nasce come strumento interno per la community Clepshydra e si trova attualmente in una fase di sviluppo attivo (V1), con una V2 pianificata che introduce un sistema torneo Swiss completo, un database relazionale e un'architettura a layer separati.

2. Infrastruttura e Ambiente di Esecuzione

2.1 — Oracle Cloud Infrastructure (OCI)

Provider	Oracle Cloud Infrastructure — Free Tier
Regione	eu-milan-1 (Availability Domain: AD-1, Fault Domain: FD-2)
OCID Istanza	ocid1.instance.oc1.eu-milan-1.anwgsljrtzu6yzqc...
Data di avvio	4 Maggio 2026, 20:29:36 UTC
IP Pubblico	80.225.91.142
Username SSH	ubuntu
Accesso	SSH con chiave privata

2.2 — Specifiche Hardware

Shape	VM.Standard.E2.1.Micro
OCPU	1 (non ridimensionabile)
RAM	1 GB
Rete	0.5 Gbps
Storage	Block Storage only (nessun disco locale)
Crittografia	In-transit encryption abilitata
Firmware	UEFI_64
Launch Mode	PARAVIRTUALIZED

2.3 — Sistema Operativo

OS	Canonical Ubuntu 24.04 LTS
Immagine	Canonical-Ubuntu-24.04-2026.03.31-0
Secure Boot	Disabilitato
Measured Boot	Disabilitato
TPM	Disabilitato
IMDS	Version 2 only

Nota: con 1 GB di RAM il bot è progettato per minimizzare il consumo di memoria. Questo vincolo architetturale ha guidato la scelta di SQLite over PostgreSQL e l'uso di cache JSON leggere invece di soluzioni in-memory come Redis.

3. Stack Tecnologico

3.1 — Dipendenze Python

discord.py	Libreria principale per l'integrazione con l'API Discord. Gestisce slash commands, modal, select menu, eventi e cog.
aiohttp	Client HTTP asincrono per le chiamate all'API Scryfall. Usato con retry logic e rate limiting.
Pillow (PIL)	Generazione immagini per il deck showcase: compositing delle card art, sfondi, watermark logo, testi.
python-dotenv	Caricamento variabili d'ambiente dal file .env.
SQLAlchemy	(Pianificato V2) ORM asincrono per SQLite. Già incluso nel requirements.
aiosqlite	(Pianificato V2) Driver asincrono per SQLite.

3.2 — Servizi Esterni

Discord API	Gateway WebSocket per ricezione eventi + REST API per invio messaggi, embed, file.
Scryfall API	POST /cards/collection per fetch batch di carte. GET prints_search_uri per verifica legalità Arena. Rate limit: 100ms tra richieste.

3.3 — Versioni

Python	3.12+
discord.py	Latest stable
Bot Version	1.0.0

4. Struttura del Repository (V1 Attuale)

```
clepshydrabot/
■■■ main.py # Entry point del bot
■■■ requirements.txt
■■■ .env # Variabili d'ambiente (non in git)
■
■■■ cogs/ # Moduli Discord (Cog)
■ ■■■ logger.py # Sistema di logging su canale Discord
■ ■■■ presentation.py # Censimento utenti e gestione ruoli
■ ■■■ tournament.py # Verifica mazze Artisan
■ ■■■ spg_override_updater.py # Aggiornamento override rarità SPG
■
■■■ utils/ # Moduli di supporto
■ ■■■ card_cache.py # Cache persistente carte (JSON)
■ ■■■ arena_overrides.py # Override rarità Arena vs Paper
■ ■■■ deck_image_generator.py # Generazione immagini mazzo
■
■■■ data/ # Dati persistenti
■■■ card_cache.json # Cache carte Scryfall
■■■ arena_rarity_data.json # Override SPG processati
■■■ cards.txt # Banlist formato Artisan
```

4.1 — Descrizione dei File Principali

main.py	Entry point. Inizializza il bot, carica tutti i cog dalla cartella cogs/, avvia il task di salvataggio periodico della cache, sincronizza i comandi slash con il server Discord.
cogs/logger.py	Cog dedicato al logging. Espone <code>send_log(level, event, user, channel, info)</code> che invia embed colorati su un canale Discord dedicato. Livelli: INFO, WARN, ERROR, DEBUG.
cogs/presentation.py	Gestisce il flusso di presentazione dei nuovi utenti. Modal con validazione regex di nome, anno nascita, formati, colori, gilde. Rimozione ruolo 'Viandante' e assegnazione 'Planeswalker'.
cogs/tournament.py	Verifica legalità dei mazze Artisan. Parsing della decklist, check banlist, fetch Scryfall, validazione rarità per game:arena, generazione immagine showcase.
utils/card_cache.py	Cache in-memory + persistenza JSON. Salvataggio asincrono ogni 60 secondi con lock per evitare race conditions. Include il campo <code>artisan_legal</code> per evitare ri-fetch.
utils/arena_overrides.py	Gestisce le carte con rarità diversa tra Arena e Paper (es. SPG). Cache in-memory con invalidazione esplicita. Funzione <code>update_spg_overrides()</code> per aggiornamento automatico via Scryfall.
utils/deck_image_generator.py	Genera l'immagine showcase del mazzo: sfondo per gilda/colore, griglia carte con immagini Scryfall, sezione sidebar, watermark logo Clepshydra colorato dinamicamente.
data/cards.txt	Banlist del formato Artisan. Una carta per riga. Include tutte le carte Alchemy (prefisso A-) e carte specifiche come Cauldron Familiar, Mishra's Bauble, ecc.

5. Funzionalità Implementate

5.1 — Sistema Presentazioni

Quando un nuovo utente entra nel server Discord riceve automaticamente il ruolo 'Viandante' che limita la visibilità dei canali. Tramite il comando `/presentati` viene aperto un modal Discord con 5 campi:

- **Dati Personali:** Nome, Nickname Arena, Anno di nascita, Professione, Provenienza
- **Inizio Magic:** Anno inizio Magic cartaceo (min 1993), Anno inizio MTGA (min 2017)
- **Preferenze:** Formati preferiti, Colori, Gilde — validati contro liste predefinite
- **Risultati:** Campo opzionale — tornei vinti, piazzamenti, ecc.
- **Altre Passioni:** Campo opzionale — interessi al di fuori di Magic

Il sistema valida ogni campo con regex e logica di dominio. In caso di errori restituisce un messaggio dettagliato con i campi mancanti e quelli con valori non validi, in ordine alfabetico. Se la validazione passa, viene pubblicato un embed nel canale presentazioni, rimosso il ruolo 'Viandante' e assegnato 'Planeswalker'.

5.2 — Verifica Mazzo Artisan

Il comando `/artisan_check_deck` apre un modal dove l'utente incolla la propria decklist nel formato Arena. Il sistema esegue i seguenti controlli in sequenza:

- **Step 1:** Parsing della decklist con supporto al blocco 'About' per il nome del mazzo
- **Step 2:** Check banlist primario — se una carta è bannata il processo si interrompe immediatamente senza chiamate Scryfall
- **Step 3:** Fetch batch via POST `/cards/collection` (chunk da 75 carte)
- **Step 4:** Per ogni carta: verifica legalità Artisan su Arena (rarietà common/uncommon, escludendo `set_type=alchemy`)
- **Step 5:** Controllo conteggio: mainboard ≥ 60 , sideboard ≤ 15
- **Step 6:** Generazione immagine showcase se il mazzo è valido
- **Step 7:** Pubblicazione embed con risultato, carte illegali e immagine nel canale dedicato

5.3 — Generazione Immagini Mazzo

Per ogni mazzo valido viene generata un'immagine PNG che include:

- Sfondo dinamico basato sulla gilda/colori del mazzo (cartelle `assets/backgrounds/`)
- Griglia delle carte del mainboard raggruppate per tipo (Creature, Spell, Land)
- Colonna sideboard con immagini e lista testuale
- Header con nome del mazzo ('About' block dalla decklist) e nickname del giocatore nel formato '(nome deck) by (player)'
- Watermark logo Clepshydra in basso a destra nella zona sideboard, colorato dinamicamente con il colore dominante dello sfondo campionato pre-oscuramento

5.4 — Sistema di Logging

Tutti gli eventi rilevanti vengono loggati su un canale Discord dedicato tramite embed colorati. I livelli disponibili sono:

- **■ INFO:** Operazioni completate con successo (presentazioni, verifiche ok)

- ■ **WARN:** Validazioni fallite, carte bannate, campi non validi
- ■ **ERROR:** Errori critici, problemi di permessi, eccezioni non gestite
- ■ **DEBUG:** Startup del bot, sincronizzazione comandi

5.5 — Override Rarità SPG

Alcune carte del set SPG (Special Guests) appaiono su Arena con rarità rare/mythic ma esistono in versione common/uncommon su carta. Il comando `/update_spg_overrides` (admin only) scansiona l'intero set via Scryfall e aggiorna automaticamente il file `arena_rarity_data.json`. La cache in-memory viene invalidata prima dell'aggiornamento per garantire coerenza.

6. Sistema di Cache e Dati Persistenti

6.1 — card_cache.json

Cache persistente delle carte Scryfall. Ogni entry contiene i dati base della carta (nome, tipo, CMC, image_url, prints_search_uri) più il campo artisan_legal (bool) aggiunto dopo il primo check. Il salvataggio avviene ogni 60 secondi in modo asincrono tramite un lock per evitare race conditions, usando un file temporaneo (.tmp) per garantire atomicità.

```
{ "lightning bolt": { "name": "Lightning Bolt", "type_line": "Instant",  
  "cmc": 1.0, "image_url": "https://...", "artisan_legal": true } }
```

6.2 — arena_rarity_data.json

Memorizza gli override di rarità per le carte con discrepanza Arena/Paper. Include la lista dei set già processati per evitare ri-scansioni. La cache in-memory viene popolata al primo accesso e invalidata esplicitamente tramite invalidate_override_cache().

6.3 — cards.txt (Banlist)

Lista delle carte bannate nel formato Artisan. Una carta per riga. Include tutte le carte Alchemy (prefisso A-) per escluderle dal formato, più carte specifiche bannate per ragioni competitive. Le carte bifronte sono indicizzate sia per nome completo che per solo fronte. La banlist viene caricata in un set Python (lookup O(1)) all'avvio del modulo.

7. Interazione con API Esterne

7.1 — Strategia Scryfall

Endpoint batch	POST /cards/collection — recupera fino a 75 carte per richiesta riducendo il numero di chiamate HTTP.
Endpoint prints	GET prints_search_uri filtrato con game:arena — restituisce solo le stampe Arena evitando falsi negativi per carte con molte stampe paper.
Rate limiting	Semaforo asyncio (1 richiesta alla volta) + delay minimo 110ms tra richieste, coerente con le linee guida Scryfall.
Retry logic	Backoff esponenziale su 429 (Retry-After header) e timeout. Massimo 5 tentativi per richiesta.
Cache	Carta già vista → nessuna chiamata Scryfall. Campo artisan_legal in cache → nessun GET prints. Riduce il traffico API del 90%+ per mazzi già verificati.
Alchemy filter	set_type=alchemy escluso esplicitamente dal check legalità per evitare falsi positivi (le carte Alchemy non sono ammesse in Artisan).

8. Flusso di una Verifica Mazza Artisan

1	Utente	Incolla la decklist nel modal /artisan_check_deck
2	Parser	Estrae nome mazzo (blocco About), lista carte con quantità, separazione mainboard/sideboard. Strip set code (ONE) 123 dal nome.
3	Banlist	Controlla ogni carta contro il set _BANLIST (O(1)). Se trovata → embed errore immediato, nessuna chiamata Scryfall.
4	Scryfall Fetch	POST /cards/collection con tutte le carte non in cache. Salva risultati in card_cache.json.
5	Artisan Check	Per ogni carta: 1) override locale, 2) cache disco (artisan_legal), 3) cache memoria (oracle_id), 4) GET prints Arena filtrato.
6	Validazione	Conta mainboard (≥ 60) e sideboard (≤ 15). Raccoglie carte illegali e carte non trovate.
7	Immagine	Se tutto ok: download immagini carte, compositing su sfondo, watermark logo, generazione PNG.
8	Pubblicazione	Embed con esito, campi errore (se presenti), immagine allegata → canale deck log del server.

9. Limitazioni Attuali e Debito Tecnico

9.1 — Architetture

- Business logic e UI Discord mescolate nello stesso file (es. tournament.py contiene parsing, API calls, embed construction)
- Configurazione distribuita: variabili d'ambiente lette in più file invece di un punto centralizzato
- Assenza di database: tutti i dati sono in file JSON flat, senza relazioni né query
- Nessun sistema torneo: gestione manuale dei pairing, risultati e standings
- Banlist in file txt: nessuna gestione per formato, nessuna UI per aggiornamenti

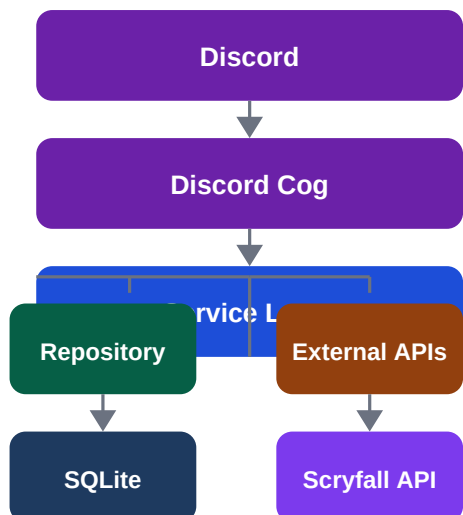
9.2 — Operativi

- Nessun Docker: deploy manuale tramite SSH, nessun riavvio automatico in caso di crash
- Nessun CI/CD: nessun test automatico, nessun workflow GitHub Actions
- Nessun versioning semantico formale
- Sistema presentazioni fragile: validazione regex su testo libero, soggetta a errori degli utenti
- 1 GB RAM: vincolo che limita le scelte tecnologiche (no PostgreSQL, no Redis)

10. Architettura V2

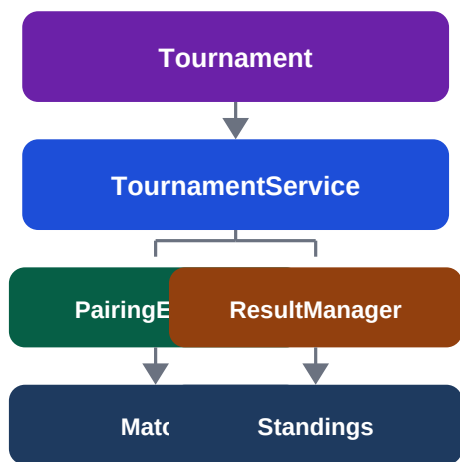
10.1 — Layer Architecture

L'architettura target di ClepshydraBot V2 segue una separazione netta in quattro layer. Ogni layer dipende solo da quello sottostante, rendendo ogni componente testabile e sostituibile indipendentemente dagli altri.



10.2 — Tournament Architecture

Il sistema torneo è il componente più complesso della V2. **TournamentService** coordina due sotto-sistemi indipendenti: **PairingEngine** per la generazione degli accoppiamenti Swiss, e **ResultManager** per la raccolta e validazione dei risultati.



11. Roadmap — ClepshydraBot V2

Sprint 0 — Git e GitHub	git init, primo commit, repository GitHub privato. Prerequisito di tutto: permette di tornare indietro in qualsiasi momento. Tempo stimato: 1 ora.
Sprint 1 — Refactor Presentazioni	Priorità numero uno. Sostituzione del modal monolitico con regex in un flusso a step: Modal (nome, nickname, anno) → Select Menu multipli (colori, gilde, formati) → Modal opzionale (risultati, passioni) → Anteprima embed → Conferma. Impatto immediato sugli utenti.
Sprint 2 — Refactor Architetture	Separazione business logic e UI Discord senza cambiare funzionalità. Introduzione di services/, config/ centralizzato, models/. Il debito tecnico identificato nel PDF viene azzerato prima di costruire il database.
Sprint 3 — SQLite	Solo dopo che l'architettura sopra è stabile. Introduzione SQLAlchemy async + aiosqlite. User, Tournament, TournamentPlayer, Match. Se in futuro si vuole migrare a PostgreSQL basta cambiare la connection string.
Sprint 4 — Sistema Tornei Swiss	La parte più interessante del progetto. /iscrivi, /avvia_torneo (admin), /risultato win loss draw X-Y, /standings. Pairing Swiss automatico, bye per dispari, ceil(log2(n)) round, punteggi 3/1/0. Diventa una piattaforma di gestione eventi.
Sprint 5 — Banlist	Il txt attuale funziona. Migrazione a tabella SQLite solo dopo Sprint 3. Gestione per formato per supportare future espansioni oltre Artisan.
Sprint 6 — GitHub pubblico	README.md professionale con architettura e screenshot, LICENSE MIT, CHANGELOG, .env.example.
Sprint 7 — Docker	Dockerfile multi-stage, docker-compose.yml con volume per data/, restart automatico in caso di crash.
Sprint 8 — CI/CD	GitHub Actions: lint (ruff), test (pytest-asyncio), build check ad ogni push e pull request.